

Incremental UB Parameterization for Stage A

1 Conventions

We fix a simple, consistent convention and use it throughout:

- Miller indices: column vector $h = (h, k, l)^\top$.
- Reciprocal-space scattering vector in the lab frame: column vector $r^* \in \mathbb{R}^3$.
- Orientation / setting matrix $A^* \in \mathbb{R}^{3 \times 3}$ defined by

$$r^* = A^* h. \quad (1)$$

- Unit cell parameters

$$\mathbf{c} = (a, b, c, \alpha, \beta, \gamma). \quad (2)$$

- A Busing–Levy type matrix $B(\mathbf{c})$ that depends *only* on the cell and maps Miller indices into reciprocal-space vectors in a crystal-fixed frame.
- A rotation $U \in SO(3)$ that maps the crystal-fixed frame into the lab frame.

With these conventions, the setting matrix is

$$A^* = U B(\mathbf{c}). \quad (3)$$

2 Baseline from DIALS/MOSFLM

From dxtbx/DIALS/MOSFLM we already have:

- A trusted initial orientation matrix A_0^* (e.g. from `crystal.get_A()`).
- A trusted initial cell

$$\mathbf{c}_0 = (a_0, b_0, c_0, \alpha_0, \beta_0, \gamma_0). \quad (4)$$

We also know a specific formula for $B(\mathbf{c})$. One common Busing–Levy style choice uses

$$\cos \alpha = \cos(\alpha), \quad \cos \beta = \cos(\beta), \quad \cos \gamma = \cos(\gamma), \quad (5)$$

and cell volume

$$V = abc \sqrt{1 + 2 \cos \alpha \cos \beta \cos \gamma - \cos^2 \alpha - \cos^2 \beta - \cos^2 \gamma}. \quad (6)$$

From these we derive reciprocal basis lengths and angles and assemble $B(\mathbf{c})$ as a lower-triangular 3×3 matrix (the standard Busing–Levy recipe).

From the baseline cell we compute

$$B_0 := B(\mathbf{c}_0), \quad (7)$$

and define the baseline orientation matrix

$$U_0 := A_0^* B_0^{-1}. \quad (8)$$

If our conventions match those of cctbx, U_0 should be numerically orthogonal:

$$U_0^\top U_0 \approx I, \quad \det U_0 \approx 1. \quad (9)$$

Thus we have the ground-truth decomposition

$$A_0^* = U_0 B_0. \quad (10)$$

Important: this decomposition is done once, at initialization.

3 UB / Quaternion Round-Trip Test

We want the UB/quaternion implementation to be internally consistent. A strict round-trip test looks like this:

1. Pick a cell $\mathbf{c}$ and build $B(\mathbf{c})$ using the same function that will be used in production.
2. Pick a rotation matrix $U \in SO(3)$. For testing, generate a random unit quaternion q and convert it to $U(q)$.
3. Form

$$A^* := UB(\mathbf{c}). \quad (11)$$

4. Run this through the UB/quaternion machinery:

- Recompute $B' := B(\mathbf{c})$.
- Compute $U' := A^* B'^{-1}$.
- Convert $U' \rightarrow q' \rightarrow U''$ using `matrix_to_quaternion` and `quaternion_to_matrix`.

5. Check the errors

$$\|B' - B(\mathbf{c})\|, \quad \|U' - U\|, \quad \|U'' - U\|. \quad (12)$$

We want relative errors at machine precision (e.g. $< 10^{-12}$ in double). If these checks fail, there is a convention bug (transpose, using B^{-1} where B was intended, wrong handedness, etc.).

This is what it means for the UB/ A^* implementation to “pass the round-trip test”.

4 Incremental Parameterization in U and Cell

Instead of making the quaternion represent the *absolute* orientation U , we let it represent a *misset* rotation ΔU on top of the baseline U_0 . Likewise, cell parameters are treated as increments around $\mathbf{c}_0$.

4.1 Orientation Increments with Quaternions

Let the optimization variable be a unit quaternion

$$q = (w, x, y, z)^\top \in \mathbb{R}^4, \quad \|q\| = 1. \quad (13)$$

We define the corresponding rotation matrix $R(q) \in SO(3)$ with the standard formula

$$R(q) = \begin{pmatrix} 1 - 2(y^2 + z^2) & 2(xy - wz) & 2(xz + wy) \\ 2(xy + wz) & 1 - 2(x^2 + z^2) & 2(yz - wx) \\ 2(xz - wy) & 2(yz + wx) & 1 - 2(x^2 + y^2) \end{pmatrix}. \quad (14)$$

The current orientation is then

$$U(q) := R(q) U_0. \quad (15)$$

Key points:

- The identity orientation (no misset) is given by

$$q_{\text{id}} = (1, 0, 0, 0)^\top, \quad R(q_{\text{id}}) = I. \quad (16)$$

- At $q = q_{\text{id}}$ we have

$$U(q_{\text{id}}) = IU_0 = U_0. \quad (17)$$

Thus orientation increments are encoded as deviations of q from $(1, 0, 0, 0)$; we never try to re-encode U_0 itself as a quaternion. In code, U_0 is a fixed 3×3 tensor and only q is trainable.

4.2 Cell Increments

We already have a baseline cell $\mathbf{c}_0$. Introduce a 6-vector of cell parameter increments

$$\delta \mathbf{c} = (\delta a, \delta b, \delta c, \delta \alpha, \delta \beta, \delta \gamma) \quad (18)$$

as trainable parameters (or log-scale versions for the lengths). For an additive model, define the current cell

$$\mathbf{c}(\delta \mathbf{c}) = (a_0 + \delta a, b_0 + \delta b, c_0 + \delta c, \alpha_0 + \delta \alpha, \beta_0 + \delta \beta, \gamma_0 + \delta \gamma). \quad (19)$$

Alternatively, we can use multiplicative deltas for the lengths, e.g.

$$a(\delta a) = a_0 e^{\delta a}, \quad \text{etc.} \quad (20)$$

The current metric matrix is then defined as a pure function of the cell:

$$B(\delta \mathbf{c}) := B(\mathbf{c}(\delta \mathbf{c})). \quad (21)$$

We never derive B from A^* ; B is always computed directly from the cell.

5 Forward Mapping: Parameters $\rightarrow$ UB $\rightarrow$ $A^* \rightarrow$ Simulator

Given refinement parameters $(q, \delta \mathbf{c})$ we build A^* as follows:

1. Normalize the quaternion:

$$\hat{q} = \frac{q}{\|q\|}. \quad (22)$$

2. Compute the miset rotation and full orientation:

$$U = R(\hat{q}) U_0. \quad (23)$$

3. Compute the current cell and B matrix:

$$\mathbf{c} = \mathbf{c}(\delta \mathbf{c}), \quad B = B(\mathbf{c}). \quad (24)$$

4. Build the setting matrix:

$$A^*(q, \delta \mathbf{c}) := UB = R(\hat{q}) U_0 B(\mathbf{c}(\delta \mathbf{c})). \quad (25)$$

This $A^*(q, \delta \mathbf{c})$ is what we convert into `(mosflm_a_star, mosflm_b_star, mosflm_c_star)` tuples and pass to nanoBragg / the simulator.

5.1 Zero-Point Behaviour

At the zero point of the parameters,

$$q = q_{\text{id}} = (1, 0, 0, 0), \quad \delta \mathbf{c} = 0, \quad (26)$$

we have

$$R(q_{\text{id}}) = I, \quad (27)$$

$$U(q_{\text{id}}) = U_0, \quad (28)$$

$$\mathbf{c}(0) = \mathbf{c}_0, \quad (29)$$

$$B(0) = B(\mathbf{c}_0) = B_0, \quad (30)$$

and therefore

$$A^*(q_{\text{id}}, 0) = U_0 B_0 = A_0^*. \quad (31)$$

This equality holds by construction, not by an approximate quaternion round-trip.

Along this forward path we never factor A^* back into (U, B) . We always go

$$(q, \delta \mathbf{c}) \longrightarrow (U, B) \longrightarrow A^* \longrightarrow \text{simulator}. \quad (32)$$

All potentially lossy or convention-sensitive operations such as $A_0^* \rightarrow (U_0, B_0)$ and `matrix_to_quaternion` are restricted to

- a one-time decomposition $A_0^* = U_0 B_0$ at initialization, and
- a pure forward map $q \rightarrow R(q)$ during refinement.

6 Before vs After: Original vs Improved Formulation

This section contrasts the original Stage-A quaternion/U-matrix formulation with the improved incremental UB parameterization.

6.1 Original Formulation (“Before”)

Conceptual pipeline:

1. Start from mapping A_0^* (from DIALS/MOSFLM).
2. Decompose into (U_0, B_{ideal}) such that $A_0^* \approx U_0 B_{\text{ideal}}$ via some helper (e.g. `derive_u_matrix_from_mosflm_a_`).
3. Encode the *absolute* orientation as a quaternion: $q_0 = \text{matrix_to_quaternion}(U_0)$. The trainable parameter q represents the full orientation, not a misset.
4. Forward pass:
 - Normalize q to $\hat{q}$.
 - Compute $U(q) = \text{quaternion_to_matrix}(\hat{q})$.
 - Rebuild $A_{\text{recon}}^* = U(q) B_{\text{ideal}}$.
 - Convert columns of A_{recon}^* to `(mosflm_a_star, mosflm_b_star, mosflm_c_star)` and pass to the simulator.
5. At the nominal “zero” point, the code implicitly expects $A_{\text{recon}}^*(q = q_0) \approx A_0^*$.

In practice, this round trip often fails due to

- inconsistent B conventions (e.g. recomputing B with a different formula),
- A vs. A^* or transpose mismatches,
- quaternion convention mismatches,
- float32/float64 and Python tuple conversions.

As a result, the orientation and hence A^* are wrong even at “zero” parameters, causing large discrepancies in predicted peak positions and catastrophic χ^2 at initialization.

6.2 Improved Formulation (“After”)

The improved design treats the DIALS/dxtbx crystal state as authoritative and uses *incremental* parameters around this baseline.

Conceptual pipeline:

1. Baseline decomposition (once):
 - Get A_0^* and $\mathbf{c}_0$ from dxtbx/DIALS.
 - Compute $B_0 = B(\mathbf{c}_0)$ via a Busing–Levy-compatible formula.
 - Define $U_0 = A_0^* B_0^{-1}$ so that $A_0^* = U_0 B_0$.
2. Define trainable increments:

- Orientation increment: unit quaternion q with $q = q_{\text{id}}$ at initialization.
- Cell increments: $\delta \mathbf{c}$ with $\delta \mathbf{c} = 0$ at initialization.

3. Forward pass:

- Normalize q to $\hat{q}$.
- Compute $U(q) = R(\hat{q})U_0$.
- Compute $\mathbf{c} = \mathbf{c}(\delta \mathbf{c})$ and $B = B(\mathbf{c})$.
- Build $A^*(q, \delta \mathbf{c}) = U(q) B(\mathbf{c})$ and pass its columns as (`mosflm_a_star`, `mosflm_b_star`, `mosflm_c_star`) to the simulator.

At zero parameters ($q = q_{\text{id}}, \delta \mathbf{c} = 0$) we have $A^*(0) = A_0^*$ by construction, so the mapping and refinement zero points are identical.

Advantages.

- The baseline crystal state (U_0, B_0, A_0^*) comes directly from dxtbx/DIALS and is never re-encoded through a quaternion round trip.
- Orientation and cell are parameterized as incremental perturbations ($\Delta R, \Delta \text{cell}$) around the baseline, matching how DIALS and other frameworks conceptualize refinement.
- The simulator always consumes A^* built as $U(\text{params})B(\text{params})$ in a single direction; no production code decomposes A^* back into (U, B) , eliminating round-trip ambiguity inside the optimization loop.

7 Summary

The robust path can be summarized as

$$A^*(q, \delta \mathbf{c}) = R\left(\frac{q}{\|q\|}\right) U_0 B(\mathbf{c}_0 + \delta \mathbf{c}), \quad (33)$$

with U_0 and $\mathbf{c}_0$ extracted once from DIALS/MOSFLM. All refinement happens in the incremental parameters $(q, \delta \mathbf{c})$; we never re-factor A^* back into U and B inside the optimization loop.